QUOTEMATE

Build Brief

What to build this week — detailed, plain-English

Week of 18 May 2026
Prepared for: Jon Pepper
Engineering owner: Jeph
Status: planning only — no code has been changed

Contents

(In Word: right-click this list and choose “Update Field” to fill page numbers.)

1. Introduction

This brief explains exactly what to build on QuoteMate this week, why each piece matters, what already exists in the system, and the steps involved. It is written in plain English so both the engineering side and John can follow it.

The two goals

Everything on the list serves one of two goals that John says will sell this product:

1. **Quotes that are accurate every single time.** A tradie will not trust the tool if a price is wrong even once.
2. **The "wow" moment.** The customer sees the actual product placed onto a photo of their own kitchen or wall.

The golden rule: order matters

Some items sit on top of others. If they are built in list order, work gets wasted. Build in this order:

> **Correct build order**
>
> Fix the pricing bug → Build the product catalogue (and update the price-checker in the same change) → Fixed bill of materials → Real-photo render → Rough-range estimate → Cost tracking → Follow-up system.

The one trap to remember all week

> **Read this before touching pricing**
>
> There is a safety check in the code (call it the "price-checker"). It rejects any price it cannot trace back to the database, and sends that quote to the paid inspection instead.
>
> So whenever you add a new place that prices come from, you **must** teach the price-checker about it in the **same** change. If you forget, every new quote gets thrown to inspection — the opposite of what John wants. This warning is repeated below where it applies.

How to read this document

The work is grouped into eight Work Packages (WP), shown in the order to do them. The number in brackets maps to the item numbers from John's list. Each Work Package has the same six parts:

- **Introduction** — the background and why it matters.
- **What it is** — the feature in plain words.
- **What's there now** — what already exists, so nothing is rebuilt by mistake.
- **What you build** — the actual steps.
- **Watch out** — traps and dependencies.

- **Done when** — a clear finish line so you know it works.

WP1 — Fix the pricing bug (item 7) — DO THIS FIRST

Introduction

This is the bug Jeph spotted on the call (“some pricing didn’t have a tenant id… messed up after we set up a new tenant”). It was checked against the code — it is real.

What it is

Each tradie (a “tenant”) should have their own price book. Right now some price-book rows have no tradie attached. When the system looks up a tradie’s prices and finds nothing, it quietly does the wrong thing: it grabs the oldest price book from another tradie, or falls back to a hard-coded default markup. No error is shown. A plumber could be quoted on electrician numbers and nobody would know.

What’s there now

The price-book table has a “tradie” column, but older rows are blank. The lookup skips blank rows. The estimator never checks that the price book it received actually belongs to this tradie.

What you build

1. Write a small script that lists every price-book row with no tradie, every active tradie that has no price book, and any place the wrong book could be used. This shows the damage before anything is changed.
2. Write a database migration that attaches the orphan rows to the correct tradie (or deletes dead ones), then make the tradie column required so this cannot happen again.
3. Add a hard rule in the estimator: if the price book does not match this tradie, stop and route to inspection with a logged reason — never silently fall back to a default.
4. Remove the “just grab the oldest book” fallback.

Watch out

Do this before testing anything else. Any accuracy test run while this bug exists is meaningless.

Done when

Every active tradie has exactly one correct price book, blank rows are gone, and a deliberately misconfigured tradie produces a clear error instead of a silent wrong price.

WP2 — Per-tradie Product & Materials Catalogue (items 5 + 6) — THE KEYSTONE

Introduction

Six things on the list are actually the same thing: a materials price book, brand priming (Clipsal), range within a brand (Iconic vs 2000), “they supply vs we supply”, the product photos, and the mid-chat photo options. They all need one product list that each tradie owns. Build this once, well, and the rest become small.

What it is

A table where each tradie’s real products live. For each product: the brand (Clipsal), the range/series (Iconic or 2000), the supplier (Reece, Bunnings), our price when we supply it, the cheaper install-only price when the customer supplies it, and a photo of the product.

What’s there now

There is a shared materials table with a brand name only. It has **no** range/series, **no** supplier, **no** product photo, and **no** separate “customer supplies it” price (only a yes/no flag, not a real second price). It is not per-tradie. There is already a per-tradie jobs table — copy that same pattern for materials so it stays consistent.

What you build

1. Create a new per-tradie materials/product table with: brand, range/series, supplier, our-supply price, customer-supply (install-only) price, product photo link, which tier it belongs to (Good/Better), and an on/off switch.
2. Set up a storage bucket for the product photos.
3. Change the estimator’s materials lookup so it reads this new per-tradie table on top of the shared one (the jobs lookup already does this — mirror it).
4. Extend the “preferred brand” hint so a tradie can also set a preferred range, and so the AI maps brand + range to a tier (Iconic → Better, 2000 → Good).
5. Make “customer supplies it” use the second, cheaper price, and add a note on the quote: “Customer to supply [item] — must meet safety standards.”
6. **Extend the price-checker** so it accepts prices from this new table and the customer-supply price. (This is the trap from the Introduction — not optional.)

Watch out

Step 6 is make-or-break. Skip it and every branded quote gets rejected to inspection. Treat steps 1–6 as one delivery, not separate tasks.

Done when

A tradie can have “Clipsal Iconic” and “Clipsal 2000” in their catalogue with photos and prices; a quote that uses one of them passes the price-checker and shows the right price; choosing “customer supplies” produces the cheaper install-only price with the safety note.

WP3 — Fixed bill of materials + expand the job list (items 6 + 11)

Introduction

John wants the same job to produce the same price every time, and a much bigger list of standard jobs so fewer quotes fall back to “pay for an inspection.”

What it is

A “bill of materials” is the fixed list of parts a job always needs. Right now the AI re-decides the parts for each job every time, which is why the price wobbles. We want each job to have a locked parts list.

What’s there now

The jobs catalogue exists (43 jobs, electrical + plumbing). But each job’s parts are loose suggestions inside the AI prompt, not a structured list, so consistency is not enforced.

What you build

1. Create a link table: each job → its fixed list of materials and quantities.
2. Change the estimator so it builds the quote lines from that fixed list, instead of free-associating.
3. Take John’s research (item 11 — he gathers it with Perplexity / Gemini / Firecrawl) and import it as new jobs plus their bills of materials. Default them to “off” so a tradie opts in.
4. Add a small consistency test: re-run a set of old quotes a few times and check the price barely changes. This is how you prove “consistent” instead of just saying it.

Watch out

Item 11 is **not** a coding task — it is John gathering data. The engineering is the import plus a price sanity-check before the data goes live (bad scraped prices = bad quotes). Also tell John: adding jobs does **not** “break the AI’s learning” — there is no learning here, the AI just reads the database. Safe to add.

Done when

A given job produces the same parts list and near-identical price across repeated runs, and the consistency test passes.

WP4 — Real product on the customer's photo (item 8) — the "wow"

Introduction

This is the main "wow." John's example: a customer photographs their kitchen sink with a daggy tap; we show their sink with the new tap on it.

What it is

Take the customer's own photo and place the exact product they are being quoted onto it.

What's there now

The system already edits the customer's photo with Google Gemini — that part works. But it only tells Gemini the product **name as text** (for example "Caroma Liano toilet"). Gemini then guesses what that looks like. That is why it is not reliably the real product.

What you build

1. When a quote's product links to a catalogue item from WP2, fetch that product's real photo.
2. Send that photo to Gemini alongside the customer's photo, with the instruction "match this exact product."
3. Do the same for the three sample images so all the images show the same product.

Watch out

This **cannot** be built before WP2 — it needs the product photo from the catalogue. Google Slides was a side comment on the call; ignore it, it is not needed.

Done when

For a product that has a catalogue photo, the rendered image on the customer's photo clearly shows that specific product, not a generic one.

WP5 — Rough price range when we can’t be exact (item 9)

Introduction

John: instead of “I don’t know, book an inspection,” say “this usually costs $200–$400, pay $99 to confirm.”

What it is

When the system cannot produce an exact price, give an honest range from past similar jobs.

What’s there now

Good news: the system already pulls “similar past quotes” to help the AI. You can reuse that exact machinery — you are not building it from scratch.

What you build

1. When a quote fails to get an exact price but there are enough similar past jobs, calculate a low–high range from them.
2. Add a third outcome (besides “full quote” and “inspection”): show the range plus a clear “this is indicative, $99 to confirm” and a $99 payment link.
3. Add the matching SMS wording.

Watch out

Keep the wording honest — it must clearly say “estimate, confirmed after the $99 visit.”

Done when

A vague request like “new toilet” returns a sensible range and a $99 link instead of a cold inspection push.

WP6 — Cost per quote (item 14)

Introduction

John wants to know what each quote actually costs to produce, so he knows the business makes money.

What it is

A dollar figure per quote: AI + images + SMS + voice.

What's there now

The system logs AI usage but never turns it into dollars. Nothing is summed.

What you build

1. Convert the AI usage already logged into dollars using the model rates.
2. Add the Gemini image cost — note each quote makes about **four** image calls (1 preview + 3 samples), so this is a real cost, not a rounding error.
3. Add Twilio SMS count and voice minutes.
4. Store a per-quote cost and show it in the dashboard.

Watch out

Low risk and additive — this can be done in parallel with the other work.

Done when

Each quote in the dashboard shows a total cost broken down by AI, images, SMS, and voice.

WP7 — Follow-up for non-converters (item 10)

Introduction

John: if only 30% accept, a VA should chase the other 70% — “you asked for a quote but didn’t go ahead, anything we can do?”

What it is

A system that knows who did not accept, and surfaces them for follow-up.

What’s there now

Nothing. **And** quote status is barely tracked — the data shows 0 accepted quotes because the system is not reliably recording acceptance. So today you literally cannot tell who “didn’t accept.”

What you build

1. First, make the system reliably record status: sent → viewed → accepted → paid.
2. Add a “Needs follow-up” list in the dashboard (quotes sent X days ago, not accepted) with the customer’s number and quote link, for a VA to work.
3. Later (not week one): an automatic SMS nudge before the human step.

Watch out

Step 1 is the real prerequisite. Do not promise the full follow-up system this week — ship status tracking plus the dashboard list, that is the honest slice.

Done when

A VA can open the dashboard and see exactly which customers got a quote and did not accept, with a way to contact them.

WP8 — Smaller confirmed changes (from the call)

Introduction

These came up strongly on the call, are cheap, and tie into the work above. Confirm them with John because they change the customer experience. They are not in the written 5–14 list.

What to build

- **Drop “Best”, keep Good + Better.** John said three options are confusing and not always right. Change the AI prompt to two tiers and update the quote page, SMS, and payment links. Keep the old database column (do not delete history).
- **$199 → $99 inspection.** The price is hard-coded in several places. Find every occurrence first, then move it to one setting so it is never hunted again.
- **Backup AI (Jeph’s idea).** Fine for the chat parts. Risky for the pricing step — a different AI may fail the price-checker and dump quotes to inspection. Keep pricing on the current setup until WP3’s consistency test exists to prove a swap is safe.
- **Mid-chat photo options.** Real MMS does **not** work on Australian Twilio numbers. Use a WhatsApp picture, or an SMS with a link to a small “pick your look” page (built from WP2’s photos). Decide with John which.

Decisions only John can make

1. Do tradies type in their own materials catalogue, or do we scrape Reece / Clipsal and they prune it? (Same build either way — only affects onboarding effort.)
2. Confirm: drop “Best”, change $199 to $99, and the rough-range idea.
3. Mid-chat options: WhatsApp photo, or SMS plus a link page?

Honest plan for the week

Fourteen items in five days is not realistic at production quality. This is the honest split so the scope conversation with John is clear.

Will ship this week	Start, but won't fully finish
• WP1 — pricing bug fix • WP2 — product catalogue + price-checker • WP3 — bill of materials + consistency test • WP4 — real-photo render • WP6 — cost per quote • WP8 — the easy changes (Good/Better, $99)	• Mid-chat photo options (needs channel decision) • WP7 — follow-up: ship status tracking + dashboard list only • Full calendar: separate multi-week job; a "price held until [date]" countdown is the cheap urgency win if time allows

Bottom line for John: the two things that sell this — accurate prices and the photo "wow" — are both in the "will ship" column, but only if WP1 and WP2 are done first and the price-checker is extended in lockstep.